

Bài giảng R, số 1

-Thống kê mô tả cho biến định tính và biến định lượng-

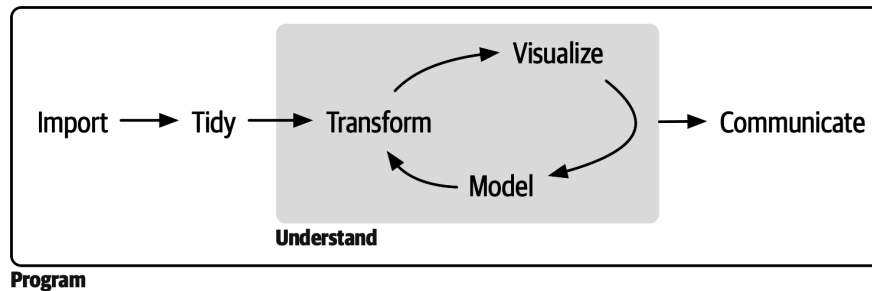
TS.Tô Đức Khánh

02/10/2023

1 Giới thiệu

Trong bài giảng này, chúng ta sẽ ôn lại các bước và các cách làm thống kê mô tả cho hai loại biến định tính và định lượng, với phần mềm R. Cụ thể hơn, ta sẽ học cách sử dụng các thư viện **tidyverse**, **ggplot2** để đọc dữ liệu từ file, thực hiện các phép toán tổng hợp dữ liệu, vẽ biểu đồ histogram, boxplot, biểu đồ hình tròn. Ngoài phần mềm R, chúng ta còn có các phần mềm hữu dụng cho việc phân tích dữ liệu, ví dụ như python, Julia. Tuy nhiên, trong khuôn khổ học phần này, chúng ta sẽ chỉ học và thành thạo R trước nhất. Sau đó, việc chuyển đổi sang các phần mềm khác sẽ không gặp nhiều trở ngại. Tài liệu chính của bài học này, tới từ cuốn sách “R for data science: Import, Tidy, Transform, Visualize and Model data” của nhóm tác giả Hadley Wickham, Mine Çetinkaya-Rundel và Garrett Grolemund. Bản online của cuốn sách có thể truy cập theo địa chỉ sau: <https://r4ds.hadley.nz/>

Một quy trình đọc và xử lý số liệu thống kê được mô tả như trong Hình 1.



Hình 1: Quy trình phân tích xử lý số liệu.

Cụ thể:

1. *import* - đọc/nhập dữ liệu từ tệp tin lưu trữ (có thể là tệp .txt, hoặc .csv, hay .xls) vào trong không gian làm việc của R;
2. *tidy* - sắp xếp dữ liệu lại sao cho gọn gàng nhằm dễ quản lý và truy xuất thông tin biến, đối tượng, chú ý, một dữ liệu gọn gàng (một cách ngắn gọn) là một dữ liệu mà mỗi một cột sẽ chứa thông tin của một biến, và mỗi dòng sẽ là thông tin của một đối tượng quan sát;
3. *transform* - Một khi dữ liệu đã được nhập và sắp xếp gọn gàng, ta có thể tạo thêm các biến mới (bằng cách áp dụng các phép biến đổi, logarithm, căn bậc hai, ...) hoặc tính các thống kê tổng hợp: trung bình, tần số, tỷ số ...
4. *visualize* - Mô tả dữ liệu (hay thống kê mô tả) là một quan trọng, giúp ta có một cái nhìn tổng quát về dữ liệu, nhằm phát hiện ra các xu hướng (có thể là đã được dự đoán hoặc chưa được dự đoán từ trước), qua đó giúp ta có định hướng được các phân tích tiếp theo nhằm giải quyết câu hỏi nghiên cứu ban đầu. Phần mô tả này có thể được hoàn thành bởi việc sử dụng bảng tổng hợp, hoặc các biểu đồ.

5. *model* - Một khi ta đã xác định rõ câu hỏi nghiên cứu, ta sẽ cần các mô hình thống kê để tìm ra câu trả lời. Mô hình ở đây có thể là mô hình kiểm định hay mô hình hồi quy. Chú ý rằng, mọi mô hình đều yêu cầu những giả định nhất định, ví dụ, giả định về phân phối chuẩn. Do đó, ta nhất thiết phải kiểm tra những giả định này. Một khi giả định không thỏa, mô hình hiện tại cần được thay đổi, hoặc ta có thể xem xét tới các phép biến đổi dữ liệu để thỏa được giả định.
6. *communicate* - Bước cuối cùng trong công đoạn phân tích dữ liệu là trao đổi, đánh giá kết quả thu được sau quá trình phân tích.

2 Nhập dữ liệu và kiểm soát dữ liệu

2.1 Nhập dữ liệu từ tệp .csv

Trong phần này, ta sẽ làm quen với cách đọc một tệp dữ liệu dạng .csv bằng hàm `read_csv()` trong thư viện `readr` một trong những thư viện được đính kèm trong thư viện `tidyverse`.

```
library(tidyverse)
```

Cấu trúc cơ bản của một đoạn code R đọc tệp dữ liệu sẽ gồm hai phần, như sau:

```
ten_data_trong_R <- read_csv("duong_dan_toi_tep_du_lieu")
```

- phần bên trái dấu "`<-`" là tên của dữ liệu được lưu lại trong không gian làm việc của R;
- phần bên phải dấu "`<-`" là cú pháp để nhập dữ liệu với đường dẫn cụ thể tới tệp dữ liệu, chú ý, đường dẫn này có thể là một đường liên kết tới một địa chỉ online;
- dấu "`<-`" là toán tử gán giá trị vào trong một biến.

Ví dụ, ta xét bộ dữ liệu `students.csv` về thực phẩm và chế độ ăn của 7 học sinh.

	Student ID	Full Name	favourite.food	mealPlan	AGE
1	1	Sunil Hufmann	Strawberry yoghurt	Lunch only	4
2	2	Barclay Lynn	French fries	Lunch only	5
3	3	Jayendra Lyne	N/A	Breakfast and lunch	7
4	4	Leon Rossini	Anchovies	Lunch only	
5	5	Chidiegwu Dunkel	Pizza	Breakfast and lunch	five
6	6	Güvenç Attila	Ice cream	Lunch only	6

Hình 2: Dữ liệu về thực phẩm và chế độ ăn của 7 học sinh.

```
data_students <- read_csv("datasets/students.csv")
```

```
## Rows: 6 Columns: 5
## -- Column specification -----
## Delimiter: ","
## chr (4): Full Name, favourite.food, mealPlan, AGE
## dbl (1): Student ID
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Để nhìn thấy bảng dữ liệu mới được nhập vào, ta có thể gõ trực tiếp tên của biến lưu dữ liệu, trong trường hợp này là `data_students`, hoặc có thể dùng cú pháp `View(data_students)`. Ví dụ:

```
data_students
```

```
## # A tibble: 6 x 5
##   `Student ID` `Full Name`   favourite.food   mealPlan      AGE
##       <dbl> <chr>           <chr>          <chr>      <chr>
## 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
## 2         2 Barclay Lynn   French fries    Lunch only      5
## 3         3 Jayendra Lyne  N/A            Breakfast and lunch 7
## 4         4 Leon Rossini    Anchovies      Lunch only      <NA>
## 5         5 Chidiegwu Dunkel Pizza          Breakfast and lunch five
## # i 1 more row
```

Từ kết quả, ta thấy rằng, dữ liệu gồm có 6 dòng (tức là 6 quan sát) và 5 cột (tương ứng với 5 biến), đã được nhập vào đúng như trong tệp .csv gốc, với tên của các cột được hiển thị ở dòng trên của mỗi cột, theo sau là viết tắt của dạng biến, ở đây:

- `<dbl>` là viết tắt của "double", tức là dạng số (bao gồm cả số thực và số nguyên);
- `<chr>` là viết tắt của "character" tức là dạng chữ.

Quan sát cột đồ ăn ưa thích `favourite.food`, ta dễ dàng nhận thấy một giá trị N/A, nó có thể là ký hiệu cho một giá trị bị khuyết (tức là không được ghi nhận lại). Trong R, giá trị bị khuyết của một biến sẽ được ký hiệu chuẩn là NA, tất cả các ký hiệu khác đều sẽ được coi là dạng chữ (nếu ký hiệu đó là chữ), và trong trường hợp này là N/A. Để hiệu chỉnh lỗi dạng này, ta có thể hiệu chỉnh trực tiếp trên tệp dữ liệu, hoặc có thể hiệu chỉnh bởi đối số `na = ...` trong hàm `read_csv()`. Một cách mặc định, đối số `na = c("", "NA")` tức là sẽ chỉ định ô trống (""), hoặc "NA" là dữ liệu khuyết, do đó, ta chỉ cần cập nhật thêm trường hợp vào trong `na = ...`, tức là `na = c("", "NA", "N/A")`. Như vậy, đoạn đọc dữ liệu được sửa lại như sau:

```
data_students <- read_csv("datasets/students.csv", na = c("", "NA", "N/A"))
```

```
## Rows: 6 Columns: 5
## -- Column specification -----
## Delimiter: ","
## chr (4): Full Name, favourite.food, mealPlan, AGE
## dbl (1): Student ID
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

và ta thu được

```
data_students
```

```
## # A tibble: 6 x 5
##   `Student ID` `Full Name`   favourite.food   mealPlan      AGE
##       <dbl> <chr>           <chr>          <chr>      <chr>
## 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
## 2         2 Barclay Lynn   French fries    Lunch only      5
## 3         3 Jayendra Lyne  <NA>           Breakfast and lunch 7
## 4         4 Leon Rossini    Anchovies      Lunch only      <NA>
## 5         5 Chidiegwu Dunkel Pizza          Breakfast and lunch five
## # i 1 more row
```

Chú ý: Ngoài hàm `read_csv()` ta còn có các hàm để đọc tệp dữ liệu như sau:

- `read_xls()`, `read_xlsx()` hoặc `read_excel()` trong thư viện `readxl`, dùng để đọc tệp dữ liệu dạng tệp excel, đuôi .xls hoặc .xlsx;

- `read_table()` trong thư viện `readr` được dùng để đọc các tệp dữ liệu dạng bảng được lưu dưới dạng tệp có đuôi `.txt` hoặc `.dat`.

2.2 Kiểm soát tên và dạng biến

Tên của các biến trong R phải được khai báo với chữ viết thường, khoảng cách giữa các chữ phải được thay thế bằng dấu gạch chân (tạo thành từ dạng hình con rắn - *snake case*). Ví dụ, tên của biến đầu tiên ‘Student ID’ là sai định dạng tiêu chuẩn, do có chữ cái viết hoa “S” và “I”, đồng thời, có khoảng cách giữa hai từ “Student” và “ID”. Tương tự, các tên biến còn lại cũng không đúng tiêu chuẩn. Để khắc phục điều này một cách tự động, ta có thể sử dụng hàm `clean_names()` trong thư viện `janitor`

```
library(janitor)
```

với cú pháp như sau:

```
data_students <- read_csv("datasets/students.csv", na = c("", "NA", "N/A")) |>
  clean_names()

## Rows: 6 Columns: 5
## -- Column specification -----
## Delimiter: ","
## chr (4): Full Name, favourite.food, mealPlan, AGE
## dbl (1): Student ID
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Dữ liệu mới có dạng như sau:

```
data_students

## # A tibble: 6 x 5
##   student_id full_name      favourite_food meal_plan      age
##   <dbl> <chr>          <chr>          <chr>      <chr>
## 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
## 2         2 Barclay Lynn   French fries    Lunch only      5
## 3         3 Jayendra Lyne  <NA>           Breakfast and lunch 7
## 4         4 Leon Rossini    Anchovies      Lunch only      <NA>
## 5         5 Chidiegwu Dunkel Pizza          Breakfast and lunch five
## # i 1 more row
```

Có thể thấy tên của các biến đã được chỉnh sửa phù hợp với tiêu chuẩn, một cách tự động.

Chú ý: cú pháp trên là sự kết hợp của hai đoạn lệnh, đoạn đầu tiên

```
read_csv("datasets/students.csv", na = c("", "NA", "N/A"))
```

dùng để đọc tệp dữ liệu, trong khi đó, đoạn thứ 2

```
clean_names()
```

dùng để điều chỉnh lại tên của các biến trong dữ liệu, kết quả sau đó được lưu lại vào tên biến trong không gian làm việc R. Sự kết hợp này được thực hiện thông qua toán tử `|>`. Sự kết hợp như thế này sẽ được dùng phổ biến trong khâu đọc, chỉnh sửa và các thao tác khác trên bộ dữ liệu (lọc, tính toán các thống kê tổng hợp). Khi ta sử dụng `|>` có nghĩa là ta đang thực hiện ghép nối các kết quả từ bên trái sang phép toán bên phải, ví dụ, `x |> f(y)` sẽ tương đương với `f(x, y)`, và `x |> f(y) |> g(z)` sẽ tương đương là `f(x, y) |> g(z)` tức là `g(f(x, y), z)`.

Bước tiếp theo, ta cần phải kiểm soát các loại biến có mặt trong dữ liệu. Như đã, nói ở phần trên, thể loại biến được thể hiện ở dòng tiếp theo tên biến, trong ví dụ này, ta có 1 biến với dạng số `<dbl>` và 4 biến dạng

chữ <chr>. Trong số các biến dạng chữ, ta dễ nhận dàng nhận thấy rằng biến `age` đang bị định dạng sai kiểu, nó đáng ra phải là dạng số <dbl>. Quan sát kỹ hơn, ta nhận thấy có một giá trị `five` thay vì 5. Như vậy, ta cần hiệu chỉnh là biến `age` mà không tác động tới tệp dữ liệu nguồn.

```
data_students <- data_students |>
  mutate(age = parse_number(if_else(age == "five", "5", age)))
```

Ở đoạn lệnh trên, ta dùng `mutate()` để tạo biến mới trong dữ liệu, ở đây, ta lấy tên biến mới cũng là `age`. Giá trị của biến mới này là giá trị của biến `age` cũ, ngoại trừ giá trị `five` được chuyển thành 5, bằng cách sử dụng hàm `if_else()` (xem thêm trong chương 12, bản online của sách). Cuối cùng, để chuyển đổi <chr> sang <dbl>, ta có thể dùng hàm `parse_number()`. Kết quả dữ liệu mới

```
data_students

## # A tibble: 6 x 5
##   student_id full_name      favourite_food meal_plan      age
##   <dbl> <chr>          <chr>          <chr>          <dbl>
## 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
## 2         2 Barclay Lynn   French fries    Lunch only      5
## 3         3 Jayendra Lyne   <NA>           Breakfast and lunch 7
## 4         4 Leon Rossini    Anchovies       Lunch only      NA
## 5         5 Chidiegwu Dunkel Pizza           Breakfast and lunch 5
## # i 1 more row
```

Một điểm cần lưu ý trước khi đi vào phân tích, đó là các biến dạng chữ <chr>, cần phải được chuyển đổi sang dạng nhân tố (*factor*) <fct> nếu muốn sử dụng trong các phân tích thống kê (xem chương 16, bản online của sách, để biết thêm chi tiết về dạng nhân tố trong R). Để làm được điều này, ta có thể dùng hàm `factor()`, cụ thể như sau:

```
data_students <- data_students |>
  mutate(meal_plan = factor(meal_plan))
```

Mặc định, hàm `factor()` sẽ xếp thứ tự của cấp độ của một nhân tố theo thứ tự bảng chữ cái hoặc chữ số.

```
levels(data_students$meal_plan)
```

```
## [1] "Breakfast and lunch" "Lunch only"
```

Chú ý: Để truy xuất một cột biến trong bảng dữ liệu, ta dùng cách gọi `ten_du_lieu$ten_bien`.

2.3 Bài tập

Bài tập 1: Đọc dữ liệu về nồng độ MAO trong tiểu cầu của các bệnh nhân, được ghi lại trong tệp `MAO.csv`. Sau khi đọc dữ liệu vào trong không gian làm việc của R, hãy cho biết có bao nhiêu quan sát và bao nhiêu biến được chứa trong dữ liệu, đồng thời, hãy kiểm tra xem liệu các tên biến, loại biến đã đúng theo quy chuẩn chưa? Nếu chưa hãy hiệu chỉnh lại cho đúng.

Bài tập 2: Đọc dữ liệu về cân nặng của trẻ sơ sinh được sinh ra tại trung tâm y tế Baystate Medical Center, Springfield, Mass, trong năm 1986, được lưu trong tệp `birthwt.txt`.

- Dữ liệu có bao nhiêu quan sát, và có bao nhiêu biến, loại của mỗi biến?
- Biến `race` biểu diễn chủng tộc của người mẹ, trong đó, 1 tương ứng với da trắng (white), 2 tương ứng với da đen (black), 3 là chủng tộc khác (other). Hãy tạo một biến `race` mới sao cho chứa thông tin chủng tộc theo dạng chữ: white, black, other. Gợi ý: dùng `if_else()` hoặc `factor()`.
- Thực hiện tương tự cho biến `smoke` (0 là không hút thuốc - no, 1 là có hút thuốc - yes).

Bài tập 3: Đọc dữ liệu về số ngày nằm điều trị tại bệnh viện của các bệnh nhi bị hen suyễn (asthma) ở Mỹ, được ghi lại trong tệp dữ liệu `AsthmaLOS.txt`.

- Dữ liệu có bao nhiêu quan sát, và có bao nhiêu biến, loại của mỗi biến?
- Tên của các biến đã đúng quy chuẩn chưa? Nếu chưa, hãy điều chỉnh lại.
- Biến **gender** cung cấp thông tin giới tính của các bệnh nhi, trong đó, 0 tức là nam (male), 1 tức là nữ (female). Tuy nhiên, có cả giá trị 4, đó có thể là sai số hoặc ám chỉ là không có thông tin. Hãy thay đổi giá trị 4 này thành giá trị khuyết NA.
- Biến **race** biểu diễn chủng tộc của bệnh nhi, trong đó, 1 tương ứng với da trắng (white), 2 tương ứng với người gốc Tây Ban Nha (hispanic), 3 là da đen (black), 4 là chủng tộc châu Á hoặc Thái Bình Dương Hải Đảo (asian/pacific islander), và 5 là chủng tộc khác (other). Hãy tạo một biến **race** mới sao cho chứa thông tin chủng tộc theo dạng chữ: white, hispanic, black, asian, other.
- Thực hiện tương tự cho biến **bed.size** miêu tả quy mô giường bệnh của bệnh viện (1 là có từ 1 tới 99 giường, 2 là có từ 100 tới 249, 3 nghĩa là có từ 250 tới 400, và 4 có nghĩa là có từ 401 tới 650).

3 Thống kê mô tả cho biến định tính

3.1 Bảng tần số, bảng tỷ số

Để miêu tả tổng hợp cho một biến định tính (có thể là định danh hoặc thứ bậc), ta có thể sử dụng bảng tần số (contingency table) hoặc bảng tỷ số (proportional table). Để tạo bảng tần số cho một biến định tính, ta sử dụng hàm `count()`, ví dụ:

```
data_students |> count(meal_plan)
```

```
## # A tibble: 2 x 2
##   meal_plan      n
##   <fct>         <int>
## 1 Breakfast and lunch      2
## 2 Lunch only              4
```

Ở đây, ta đang áp dụng hàm `count()` lên biến `meal_plan` trong dữ liệu `data_students`, kết quả thu được là bảng tần số cho loại bữa ăn trong ngày. Bảng kết quả này, về mặt kỹ thuật trong R là một bảng dữ liệu với 2 cột, tương ứng, tên thể loại và tần số. Do đó, ta có thể tạo thêm cột biến mới chứa thông tin về tỷ số của các thể loại.

```
data_students |> count(meal_plan) |> mutate(prop = n/sum(n))
```

```
## # A tibble: 2 x 3
##   meal_plan      n    prop
##   <fct>         <int>  <dbl>
## 1 Breakfast and lunch      2 0.333333
## 2 Lunch only              4 0.666667
```

Đoạn lệnh ở trên tương đương với việc ta lưu bảng tần số và áp dụng `mutate`:

```
table_meal <- data_students |> count(meal_plan)
table_meal |> mutate(prop = n/sum(n))
```

Ngoài ra, ta cũng có thể tạo bảng tần số chéo cho 2 hoặc nhiều biến định tính. Ví dụ, quan sát mẫu dữ liệu cân nặng của trẻ sơ sinh được quan sát ở Mỹ, trong tệp `birthwt.txt`.

```
data_birth <- read_table(file = "datasets/birthwt.txt")
```

```
##
## -- Column specification -----
## cols(
##   low = col_double(),
##   age = col_double(),
```

```
## lwt = col_double(),
## race = col_double(),
## smoke = col_double(),
## ptl = col_double(),
## ht = col_double(),
## ui = col_double(),
## ftv = col_double(),
## bwt = col_double()
## )
```

Ta muốn thống kê số lượng người hút thuốc và chủng tộc của họ. Trước hết, ta cần chuyển đổi các biến `race` và `smoke` thành dạng nhân tố (biến định danh) và ghép tên cho từng phân loại để dễ nhận dạng.

```
data_birth <- data_birth |>
  mutate(race = factor(race, labels = c("white", "black", "other")),
         smoke = factor(smoke, labels = c("no", "yes")))
```

Bảng tổng hợp, được hoàn thành một cách dễ dàng với hàm `count()`:

```
data_birth |> count(race, smoke)
```

```
## # A tibble: 6 x 3
##   race smoke      n
##   <fct> <fct> <int>
## 1 white no      44
## 2 white yes     52
## 3 black no      16
## 4 black yes     10
## 5 other no      55
## # i 1 more row
```

Kết quả trả ra là một bảng dữ liệu, do đó, ta có thể trình bày lại theo dạng bảng tần số hai chiều, bằng cách kết hợp với hàm `spread()`, ví dụ:

```
data_birth |> count(race, smoke) |> spread(race, n)
```

```
## # A tibble: 2 x 4
##   smoke white black other
##   <fct> <int> <int> <int>
## 1 no      44     16     55
## 2 yes     52     10     12
```

Ở dòng lệnh trên, ta tạo bảng tần số (`n`) với cột là các phân loại của `race`.

Thực hành: Hãy thử dòng lệnh dưới đây, và nhận xét sự khác biệt:

```
data_birth |> count(race, smoke) |> spread(smoke, n)
```

Một cách khác, có phần tiện lợi hơn, đó là sử dụng hàm `tabyl()` trong thư viện `janitor`, hàm này sẽ trả về kết quả là một bảng tần số cho 1 biến định tính, bảng tần số chéo của 2 biến định tính, và có thể là 3 biến định tính.

```
data_birth |> tabyl(smoke, race)
```

```
##   smoke white black other
##     no    44    16    55
##     yes   52    10    12
```

Chú ý, biến nào viết trước sẽ là biến xuất hiện ở dòng của bảng tần số chéo. Cũng bằng sử dụng `tabyl()`, ta có thể tạo ra bảng tỷ số chéo của 2 biến định tính, bằng cách ghép thêm hàm `adorn_percentages()`, với

đối số chuyển vào là "row" nếu ta muốn tính tỷ số theo dòng, hoặc "col" nếu ta muốn tính theo cột của bảng, ví dụ:

```
data_birth |> tabyl(smoke, race) |> adorn_percentages("col")
```

```
## smoke      white      black      other
##    no 0.4583333 0.6153846 0.8208955
##    yes 0.5416667 0.3846154 0.1791045
```

Thực hành: Hãy thử lại dòng lệnh trên với đối số "row"! Tìm hiểu thêm về cách dùng hàm `tabyl()` tại trang web <https://cran.r-project.org/web/packages/janitor/vignettes/tabyls.html>

3.2 Biểu đồ cột cho một biến

Trong phần này, ta sẽ học cách vẽ biểu đồ cột (*bar plot*) và biểu đồ hình tròn (*pie plot*) với thư viện `ggplot2`. Xem chi tiết về vẽ hình với `ggplot2` tại chương 2, của sách bản online. Về cơ bản, cấu trúc của một đoạn lệnh vẽ biểu đồ với `ggplot2` như sau:

```
ggplot(data = ten_du_lieu,
       mapping = aes(x = ten_bien_o_truc_x, y = ten_bien_truc_y,
                     cac_doi_so_khac)) +
  \\ các hàm vẽ đối tượng: bar_plot, histogram, boxplot +
  \\ các hàm hiệu chỉnh hình
```

Trong một số trường hợp, ta có thể chỉ sử dụng x hoặc y. `cac_thong_so_khac` có thể là:

- `fill`: tô màu bên trong của hình
- `color`: tô màu đường viền
- `shape`: hiệu chỉnh hình dạng của điểm

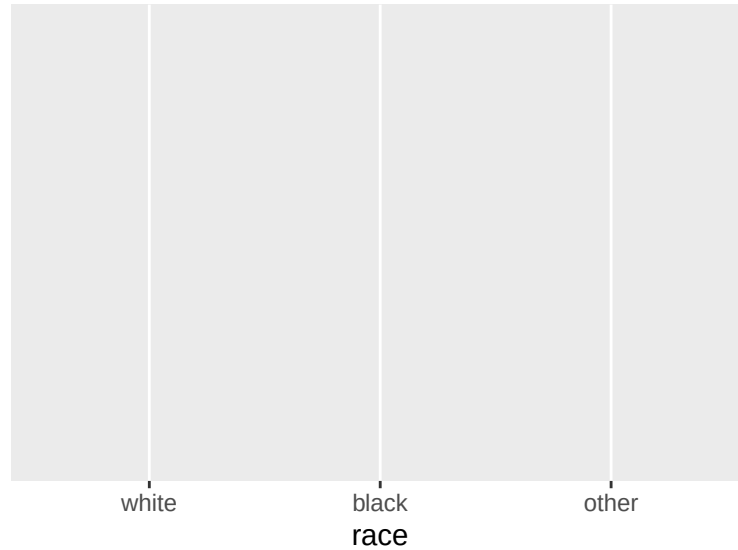
Tùy vào từng tình huống mà ta sẽ sử dụng các thông số này. Dấu "+" trong đoạn lệnh trên là để ghép nối các câu lệnh vẽ với nhau tạo thành một khối vẽ `ggplot`.

Để vẽ biểu đồ cột, ta dùng hàm `geom_bar()`. Có một số cách vẽ như sau.

Cách 1, vẽ trực tiếp từ dữ liệu Ví dụ, quan sát lại bộ dữ liệu `birthwt.txt`, ở phần trước, ta mong muốn vẽ biểu đồ cột biểu diễn số lượng các chủng tộc có trong dữ liệu. Về mặt kỹ thuật, ta thực hiện theo cách bước sau đây:

Bước 1. Sử dụng `ggplot()` để tạo một khung biểu đồ dựa trên thông tin của dữ liệu được nhập vào (`data`), với trục x được xác định thông qua đối số `mapping`.

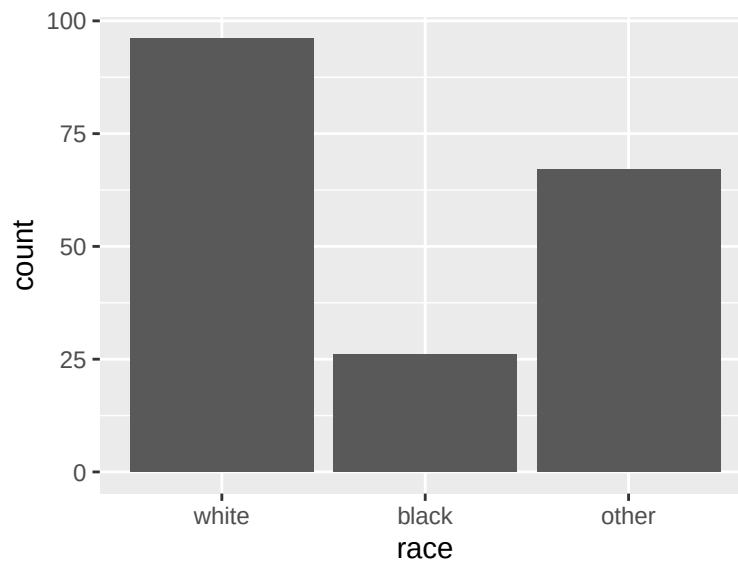
```
ggplot(data = data_birth, mapping = aes(x = race))
```

Ta thấy trục x đã được hiển thị với 3 loại chủng tộc được ghi nhận trong dữ liệu. Tuy nhiên, chưa có giá trị ở trục y được biểu diễn trong biểu đồ.

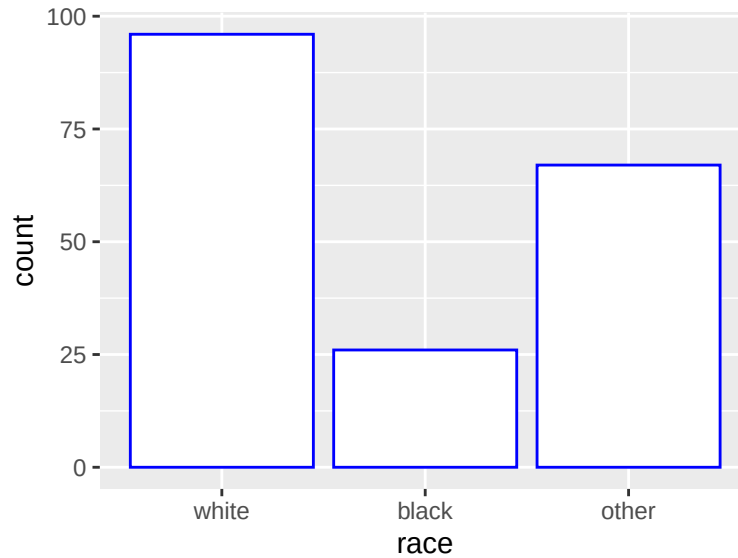
Bước 2. Tiếp theo đó, ta ghép nối hàm `geom_bar()` vào đoạn mã trên bằng dấu `+`. Kết quả thu được là biểu đồ cột cho tần số của các chủng tộc.

```
ggplot(data = data_birth, mapping = aes(x = race)) +  
  geom_bar()
```



Cú pháp `geom_bar()` tương đương với việc ta đang sử dụng các thiết lập mặc định, trong đó, tần số (biểu diễn tại trục y) được tính tự động thông qua hàm `stat_count()`, màu của các cột (đường viền và bên trong cột) cũng là mặc định. Để thay đổi màu viền của cột, ta dùng đối số `color`, và để đổi màu trong cột, ta dùng đối số `fill`. Ví dụ, ta muốn viền có màu xanh da trời (*blue*) và màu cột là trắng (*white*), đoạn lệnh vẽ sẽ như sau:

```
ggplot(data = data_birth, mapping = aes(x = race)) +  
  geom_bar(color = "blue", fill = "white")
```



Thực hành 1: Hãy ghép nối đoạn lệnh vẽ biểu đồ cột ở trên, với các đoạn lệnh sau đây, để thay đổi cách biểu diễn của biểu đồ:

- `xlab("Chủng tộc")`
- `ylab("Số lượng")`
- `theme_classic()`
- `scale_x_discrete(breaks = c("white", "black", "other"),
labels = c("Da trắng", "Da \u0111en", "Kh\u00e1c"))`

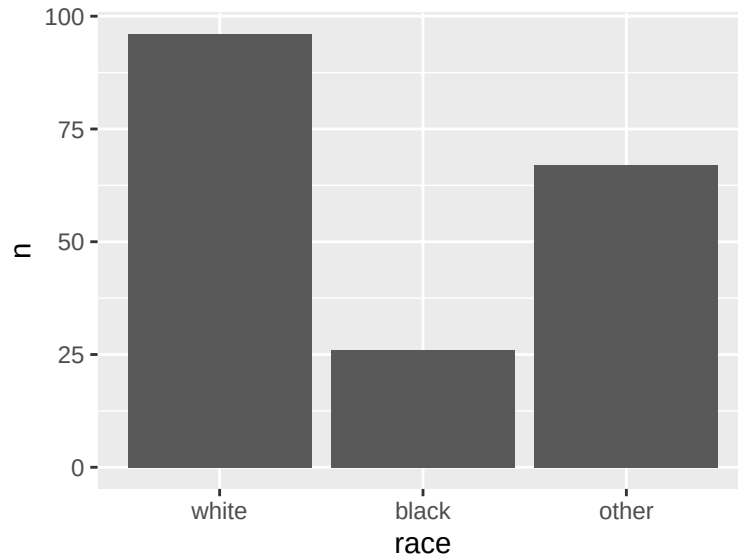
đoạn lệnh này nhằm thay thế lại nhãn của các chủng tộc được hiển thị tại trục x. Để hiển thị chữ cái tiếng Việt trong hình vẽ, ta có thể dùng mã unicode, chuyển đổi từ hoặc cụm từ tiếng Việt sang mã unicode tại địa chỉ sau: <https://www.vietnamesetools.com/vi/vietnamese-to-unicode>.

Cách 2, vẽ từ kết quả của bảng tần số Với cách này, trước tiên, ta sẽ tạo bảng tần số (hoặc tỷ số), sau đó, sử dụng `ggplot()` và `geom_bar()` tương tự như trên.

```
tab_race <- data_birth |> count(race)
tab_race
```

```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 white    96
## 2 black    26
## 3 other    67
```

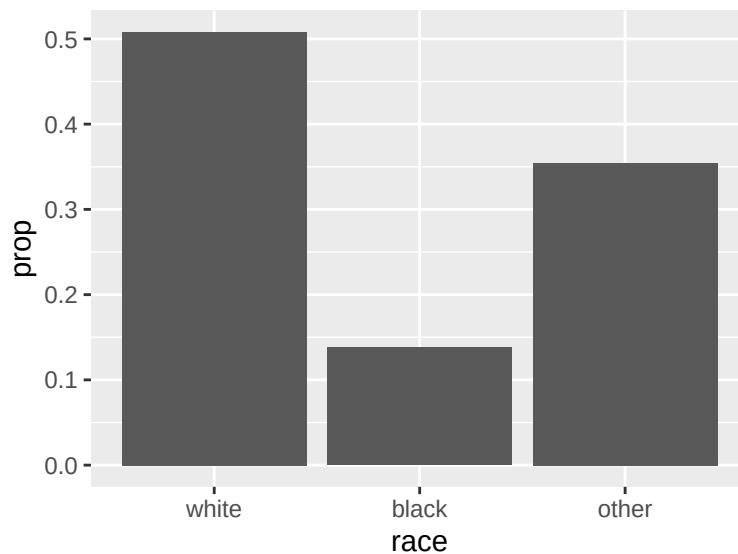
```
ggplot(data = tab_race, mapping = aes(x = race, y = n)) +  
  geom_bar(stat = "identity")
```



Chú ý, do ta đã tính tần số, nên ta cần khai báo `stat = "identity"` để chỉ định cho hàm `geom_bar()` không tính nữa, thay vào đó sử dụng kết quả `n` từ dữ liệu `tab_race`.

Để vẽ biểu đồ cột cho tỷ số của các chủng tộc, từ bảng tần số, trước hết ta cần tạo cột tỷ số, sau đó sử dụng cột này thay vì `n`

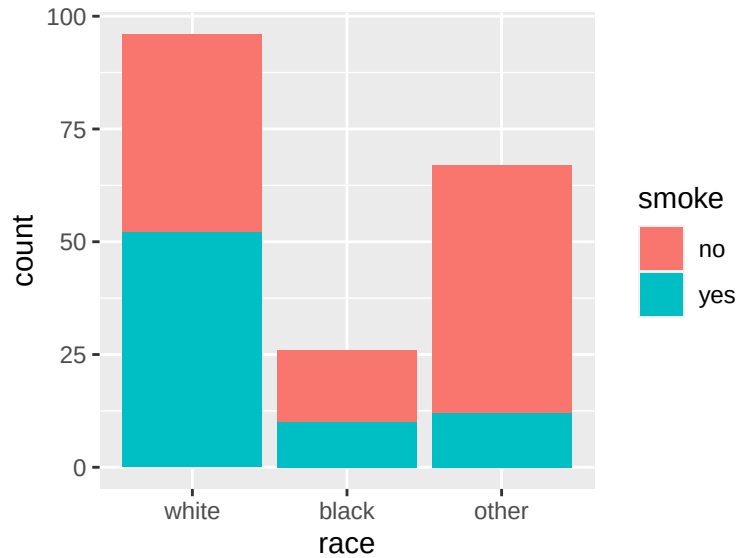
```
tab_race <- tab_race |> mutate(prop = n/sum(n))
ggplot(data = tab_race, mapping = aes(x = race, y = prop)) +
  geom_bar(stat = "identity")
```



3.3 Biểu đồ cột cho 2 biến định tính

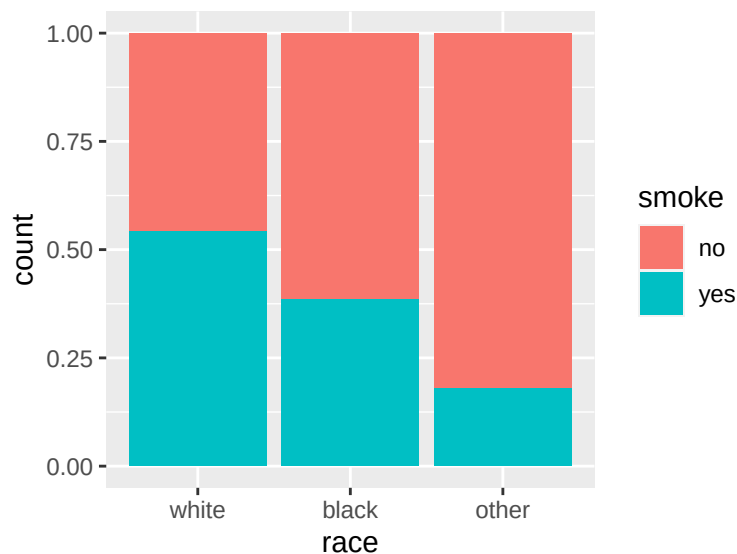
Biểu đồ cột cho 2 biến định tính dùng để mô tả khái quát mối liên hệ của hai biến này với nhau. Để vẽ, ta vẫn sử dụng hàm `geom_bar()`, nhưng với cấu trúc khác so với phần trước.

```
ggplot(data = data_birth, mapping = aes(x = race, fill = smoke)) +
  geom_bar()
```



Ý tưởng là ta vẫn vẽ các cột của biến `race`, nhưng màu của cột (tức là `fill`) sẽ được chia làm hai nửa theo số lượng `no` và `yes` của biến `smoke`. Biểu đồ dạng này sẽ khó đưa ra được nhận xét cho mối quan hệ giữa sự hút thuốc và chủng tộc. Thay vào đó, ta nên biểu diễn theo dạng tỷ số. Để làm được điều đó, ta sẽ thêm đối số `position = "fill"` vào trong hàm `geom_bar()`, như sau.

```
ggplot(data = data_birth, mapping = aes(x = race, fill = smoke)) +
  geom_bar(position = "fill")
```



Với biểu đồ mới này, ta có thể thấy được sự thay đổi về cơ cấu % người hút thuốc trong các chủng tộc, cụ thể, người da trắng có xu hướng hút thuốc nhiều hơn so với người da đen, và người thuộc các chủng tộc khác.

Thực hành 2: Sử dụng đoạn lệnh gần nhất để vẽ biểu đồ tần số, mô tả mối quan hệ của `race` và `smoke`. Và thực hiện các yêu cầu sau:

- thay đổi tên của trục y sao cho phù hợp tỷ lệ đang được trình bày;
- điều chỉnh nền của biểu đồ sang màu trắng, và không có kẻ ô lưới;
- hiệu chỉnh tên của trục x và trục y sang tiếng Việt;

- sửa dụng hàm `scale_fill_discrete()` để tùy chỉnh màu của cột (gợi ý, các đối số là tương tự với hàm `scale_x_discrete()`).

Thực hành 3: Lập lại thực hành 2, nhưng với trục `x` biểu diễn trạng thái hút thuốc của người mẹ, tức là biến `smoke`.

3.4 Bài tập

Bài tập 4: Xét bộ dữ liệu về cân nặng của trẻ sơ sinh trong tệp `birthwt.txt`. Biến `low` biểu thị nhóm cân nặng của trẻ, trong đó, 1 có nghĩa là cân nặng nhỏ hơn 2.5kg.

- Chuyển đổi biến `low` thành dạng phân loại.
- Thiết lập bảng tần số và tỷ số chéo, biểu diễn mối quan hệ giữa cân nặng thấp và chủng tộc của người mẹ.
- Thiết lập bảng tần số và tỷ số chéo, biểu diễn mối quan hệ giữa cân nặng thấp và trạng thái hút thuốc của người mẹ.
- Vẽ biểu đồ cột mô tả % các chủng tộc trong hai nhóm cân nặng của trẻ. Có thể có nhận xét nhanh được gì?
- Lập lại biểu đồ cột ở câu trên, nhưng tách ra thành hai biểu đồ theo trạng thái hút thuốc của người mẹ. (Gợi ý: ghép nối với hàm `facet_wrap(~ smoke)`). Nêu một số nhận xét.
- Sử dụng hàm `scale_fill_discrete()` để tùy chỉnh hiển thị tiếng Việt cho chú thích màu của chủng tộc (gợi ý, chỉ cần sử dụng các đối số `name` - cho tên của chú thích- và `labels` - cho tên của các chủng tộc).
- Lập lại đoạn lệnh vẽ trên, thêm đối số `labeller = labeller(smoke = c(no = "không hút thuốc", yes = "có hút thuốc"))` vào hàm `facet_wrap(~ smoke)`).

Bài tập 5: Xét bộ dữ liệu về cân nặng của trẻ sơ sinh trong tệp `birthwt.txt`. Biến `age` biểu thị tuổi của người mẹ, giao động từ 14 tới 45.

- Sử dụng hàm `cut()` để phân độ tuổi thành ba nhóm `young`, `moderate` và `old`, lưu biến mới tạo đó vào trong dữ liệu, với tên là `age_grps`. Gợi ý, cú pháp của hàm `cut()`:

```
cut(x, breaks = ..., labels = ...)
```

trong đó, `x` là biến cần chia nhóm, `breaks` là vectơ chứa điểm xác định các khoảng chia, `labels` là nhãn cho các khoảng được chia, `include.lowest` là đối số xác định xem có chứa điểm bên chia bên trái hay không (nếu đặt là `TRUE`). Ví dụ, ta muốn chia một biến `x` có giá trị từ 1 tới 15 thành 5 khoảng, `(0, 3]`, `(3, 6]`, `(6, 9]`, `(9, 12]`, `(12, 15]`:

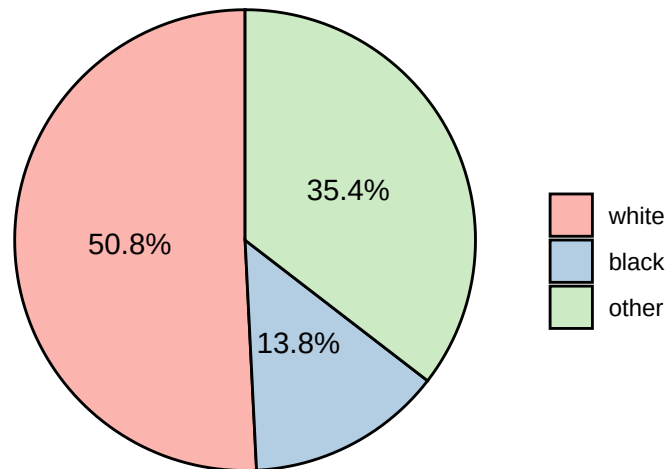
```
cut(x, breaks = c(0, 3, 6, 9, 12, 15),
    labels = c("grp_1", "grp_2", "grp_3", "grp_4", "grp_5"))
```

- Tạo bảng tần số và tỷ số cho biến `age_grps`.
- Vẽ biểu đồ cột cho biến `age_grps`, với tiêu đề bằng tiếng Việt.
- Vẽ biểu đồ để đánh giá mối liên hệ giữa nhóm cân nặng và nhóm tuổi. Nhận xét ngắn gọn về kết quả của biểu đồ.
- Việc mang thai ở độ tuổi trẻ có liên quan tới chủng tộc hay không? Hãy tìm kiếm bằng chứng bằng một biểu đồ phù hợp, với tiêu đề và chú thích bằng tiếng Việt. Hãy nhận xét về kết quả.

3.5 Biểu đồ tròn (đọc thêm)

Đoạn lệnh sau được dùng để vẽ biểu đồ hình tròn miêu tả cấu trúc (theo tỷ lệ %) của chủng tộc được ghi nhận trong dữ liệu `birthwt`.

```
data_birth |> count(race) |>
  mutate(value = 100*n/sum(n),
         csum = rev(cumsum(rev(value))),
         pos = value/2 + lead(csum, 1),
         pos = if_else(is.na(pos), value/2, pos)) |>
  ggplot(aes(x = "", y = value, fill = race)) +
  geom_col(width = 1, color = 1) +
  coord_polar(theta = "y") +
  geom_text(aes(label = paste0(round(value, 1), "%"),
                 position = position_stack(vjust = 0.5), size = 4) +
  scale_fill_brewer(palette = "Pastel1", breaks = c("white", "black", "other"),
                   labels = c("white", "black", "other")) +
  guides(fill = guide_legend(title = "")) +
  theme_void()
```



4 Thống kê mô tả cho biến định lượng

Đối với biến định lượng, ta cũng có hai cách để mô tả khái quát dữ liệu:

- sử dụng các thống kê tổng hợp: trung bình (*mean*), độ lệch chuẩn (*standard deviation*);
- sử dụng biểu đồ tần số *histogram*, biểu đồ hộp *boxplot*, biểu đồ phân tán *scatter plot*.

4.1 Thống kê tổng hợp cho một biến định lượng

Đầu tiên, ta sẽ tính giá trị trung bình và độ lệch chuẩn cho một biến định lượng trong dữ liệu.

Xét dữ liệu `birthwt.txt` về cân nặng của trẻ sơ sinh ở Mỹ. Biến `bwt` biểu thị cân nặng của trẻ khi sinh (đơn vị: grams). Để tạo một bảng báo cáo bao gồm trung bình và độ lệch chuẩn của `bwt`, ta sử dụng hàm `summarise()` như sau:

```
data_birth |> summarise(tb = mean(bwt), dlc = sd(bwt))

## # A tibble: 1 x 2
##       tb      dlc
##   <dbl> <dbl>
```

```
## 1 2944.66 729.022
```

Trong đó, ta kết hợp các hàm cơ bản của R: `mean()` và `sd()`. Ngoài ra, ta cũng có thể gọi các hàm tính trung vị (*median*), giá trị nhỏ nhất, giá trị lớn nhất, khoảng tứ phân vị trong *interquartile range*.

Thực hành 4: Tạo bảng tóm tắt cho biến `bwt`, có chứa các giá trị trung vị, giá trị lớn nhất, giá trị nhỏ nhất, khoảng tứ phân vị.

4.2 Thống kê tổng hợp cho nhiều biến định lượng

Trong các báo cáo thống kê, ta thường phải trình bày bảng thống kê tổng hợp cho nhiều biến định lượng cùng một lúc. Để làm điều này một cách nhanh chóng và có kết quả trình bày ưu nhìn, ta sẽ thực hiện theo hai bước như sau:

Bước 1. Sử dụng hàm `summarise()` kết hợp với `across()` để tính các ước lượng thống kê, kết quả sau đó được lưu lại trong một biến,

```
df_sum <- data_birth |>
  summarise(across(c("bwt", "lwt", "age"),
    list(gtnn = min, gtln = max, tv = median,
         tb = mean, dlc = sd)))
```

trong hàm `across()`, ta cần 2 đối số: đầu tiên là tên của các biến cần tính, sau đó là danh sách các hàm cần dùng để tính các ước lượng thống kê mong muốn. Ở đây, ta muốn tính giá trị nhỏ nhất (`gtnn`), giá trị lớn nhất (`gtln`), trung vị (`tv`), trung bình (`tb`) và độ lệch chuẩn (`dlc`). Bảng kết quả thô sẽ là

```
df_sum

## # A tibble: 1 x 15
##   bwt_gtnn bwt_gtln bwt_tv  bwt_tb bwt_dlc lwt_gtnn lwt_gtln lwt_tv  lwt_tb lwt_dlc
##   <dbl>    <dbl> <dbl>   <dbl>   <dbl>   <dbl>    <dbl> <dbl>   <dbl>   <dbl>
## 1      709    4990  2977 2944.66 729.022      80     250    121 129.815 30.5794
## # i 5 more variables: age_gtnn <dbl>, age_gtln <dbl>, age_tv <dbl>, age_tb <dbl>,
## #   age_dlc <dbl>
```

Bước 2. Để tạo ra một bảng kết quả dễ nhìn hơn, ta dùng đoạn lệnh sau:

```
df_stats_tidy <- df_sum |> gather(ten, gt) |>
  separate(ten, into = c("bien", "tk"), sep = "_") |>
  spread(tk, gt) |>
  select(bien, gtnn, gtln, tv, tb, dlc)
print.data.frame(df_stats_tidy, digits = 4)
```

```
##   bien gtnn gtln  tv    tb    dlc
## 1  age   14  45   23  23.24  5.299
## 2  bwt   709 4990 2977 2944.66 729.022
## 3  lwt    80  250 121 129.81 30.579
```

Ý tưởng của đoạn lệnh trên như sau:

- Sử dụng `gather()` để tạo ra một bảng dữ liệu với hai cột có tên là `ten` (tên) và `gt` (giá trị).

```
df_sum |> gather(ten, gt)
```

```
## # A tibble: 15 x 2
##   ten      gt
##   <chr>   <dbl>
## 1 bwt_gtnn  709
## 2 bwt_gtln 4990
## 3 bwt_tv   2977
```

```
## 4 bwt_tb    2944.66
## 5 bwt_dlc    729.022
## # i 10 more rows
```

- Thành phần của cột `ten` bao gồm hai thành phần, được ngăn cách nhau bởi dấu `_`, thành phần phía trước là tên biến được quan tâm, thành phần phía sau là tên của ước lượng thống kê cần tính (`gtmn`, `gtln`, `tb`, `tv`, `dlc`). Với tính chất này, ta có thể tách `ten` thành hai cột mới, đặt là `"bien"` và `"tk"`, lần lượt biểu thị tên của biến và tên của ước lượng thống kê. Để làm điều này, ta dùng hàm `separate()`:

```
df_sum |> gather(ten, gt) |>
  separate(ten, into = c("bien", "tk"), sep = "_")
```

```
## # A tibble: 15 x 3
##   bien tk      gt
##   <chr> <chr>   <dbl>
## 1 bwt   gtnn    709
## 2 bwt   gtln   4990
## 3 bwt   tv     2977
## 4 bwt   tb     2944.66
## 5 bwt   dlc     729.022
## # i 10 more rows
```

- Từ bảng mới này, ta dùng hàm `spread()` cho hai biến `tk` và `gt`, để tạo ra bảng hai chiều (tương tự như ta đã làm cho bảng tần số chéo):

```
df_sum |> gather(ten, gt) |>
  separate(ten, into = c("bien", "tk"), sep = "_") |>
  spread(tk, gt)
```

```
## # A tibble: 3 x 6
##   bien      dlc   gtln   gtnn      tb      tv
##   <chr>   <dbl> <dbl> <dbl>   <dbl> <dbl>
## 1 age    5.29868   45    14  23.2381    23
## 2 bwt    729.022  4990   709 2944.66   2977
## 3 lwt    30.5794   250    80 129.815   121
```

- Tuy nhiên, thứ tự các cột hơi lộn xộn, ta có thể sắp xếp lại theo thứ tự mà mình mong muốn, bằng cách sử dụng hàm `select()`:

```
df_sum |> gather(ten, gt) |>
  separate(ten, into = c("bien", "tk"), sep = "_") |>
  spread(tk, gt) |>
  select(bien, gtnn, gtln, tv, tb, dlc)
```

```
## # A tibble: 3 x 6
##   bien   gtnn   gtln   tv      tb      dlc
##   <chr> <dbl> <dbl> <dbl>   <dbl>   <dbl>
## 1 age    14    45    23  23.2381  5.29868
## 2 bwt    709  4990  2977 2944.66  729.022
## 3 lwt     80   250   121 129.815  30.5794
```

- Kết quả sau đó được lưu trữ vào một biến. Để in kết quả, ta dùng hàm `print.data.frame()` với đối số `digits = 4` để điều chỉnh số chữ số sau dấu thập phân được hiển thị.

Thực hành 5: Tập dữ liệu `Pima.csv` ghi nhận lại các chỉ số sinh học khác nhau của 532 nữ thổ dân da đỏ Pima, độ tuổi từ 21 trở lên, sống tại Phoenix, Arizona, Mỹ. Những người này cũng được kiểm tra bệnh đái tháo đường theo tiêu chuẩn của tổ chức y tế thế giới (WHO). Các biến được ghi nhận như sau:

- `npreg`: số lần mang thai;

- **glu**: nồng độ plasma glucose trong xét nghiệm dung nạp glucose đường uống;
- **bp**: huyết áp tâm trương (mm Hg);
- **skin**: độ dày nếp gấp da cơ tam đầu - bắp thịt to ở đẳng sau cánh tay trên (mm);
- **bmi**: chỉ số mật độ cơ thể (cân nặng (kg) chia cho bình phương chiều cao (m));
- **ped**: khả năng bị bệnh tiểu đường theo phả hệ;
- **age**: độ tuổi (năm);
- **type**: trạng thái bị bệnh tiểu đường (yes/no);

Hãy thực hiện các thao tác sau:

1. Nhập dữ liệu này vào trong không gian làm việc của R, sau đó, kiểm tra xem liệu các tên biến, loại biến đã đúng theo quy chuẩn chưa? Nếu chưa hãy hiệu chỉnh lại cho đúng.
2. Lập bảng thống kê tổng hợp cho 1 biến định lượng bất kỳ.
3. Lập bảng thống kê tổng hợp cho 1 vài biến định lượng bất kỳ.

4.3 Thống kê tổng hợp cho biến định lượng phân theo nhóm

Trong một số trường hợp, các đối tượng trong nghiên cứu có thể được phân thành các nhóm theo một tính chất hay biến định lượng nào đó, ví dụ:

- giới tính;
- nhóm tuổi;
- tình trạng bệnh;
- trạng thái hút thuốc lá.

Khi đó, các ước lượng thống kê tổng hợp của biến định lượng cũng có thể sẽ khác nhau theo từng nhóm đối tượng. Trong phần này, ta sẽ học cách tạo ra một bảng thống kê tổng hợp cho 1 hoặc nhiều biến định lượng, khi đối tượng được phân theo nhóm.

Đối với trường hợp 1 biến định lượng, ta vẫn sẽ sử dụng hàm `summarise()`, tuy nhiên, trước đó, ta sẽ dùng hàm `group_by()` để phân đối tượng thành các nhóm dựa trên thông tin của một hoặc hai biến phân loại. Ví dụ, ta muốn tạo bảng thống kê tổng hợp, bao gồm thông tin của số lượng đối tượng, trung bình và độ lệch chuẩn, cho `bwt` cho các nhóm chủng tộc khác nhau được ghi nhận trong dữ liệu `birthwt.txt`, cấu trúc đoạn lệnh sẽ như sau:

```
data_birth |> group_by(race) |>
  summarise(n = n(), tb = mean(bwt), dlc = sd(bwt))
```

```
## # A tibble: 3 x 4
##   race      n    tb    dlc
##   <fct> <int> <dbl> <dbl>
## 1 white    96 3103.74 727.724
## 2 black    26 2719.69 638.684
## 3 other    67 2804.01 721.301
```

trong đó, `group_by(race)` sẽ phân chia đối tượng thành ba nhóm: `white`, `black` và `other`. Hàm `n()` dùng để đếm số lượng đối tượng.

Thực hành 6: thực hiện các thao tác sau

- tạo bảng thống kê tổng hợp cho biến `bwt` theo các nhóm của trạng thái hút thuốc của mẹ;
- tạo bảng thống kê tổng hợp cho biến `bwt` theo các nhóm của chủng tộc và trạng thái hút thuốc của mẹ;

- lập lại các bảng trên, nhưng thêm một cột tính độ lệch chuẩn của trung bình mẫu.

Trong trường hợp, ta muốn làm bảng tổng kê tổng hợp cho nhiều hơn một biến định lượng, ta cũng sẽ làm tương tự như trường hợp không phân nhóm. Lưu ý, khi tạo bảng kết quả, hàm `gather()` sẽ cần chỉnh sửa để không sử dụng cột chứa thông tin của nhóm được chia. Cụ thể, trong trường hợp này, cột `race` sẽ không được dùng, khi đó cấu trúc `gather()` sẽ là `gather(ten, gt, -race)`, dấu `"-"` có nghĩa là “loại bỏ”.

```
df_sum_race <- data_birth |> group_by(race) |>
  summarise(across(c("bwt", "lwt", "age"),
    list(gttn = min, gtln = max, tv = median,
      tb = mean, dlc = sd)))
df_stats_tidy_race <- df_sum_race |> gather(ten, gt, -race) |>
  separate(ten, into = c("bien", "tk"), sep = "_") |>
  spread(tk, gt) |>
  select(race, bien, gttn, gtln, tv, tb, dlc)
print.data.frame(df_stats_tidy_race, digits = 4)
```

```
##   race bien gttn gtln   tv    tb    dlc
## 1 white age   14  45  23.5  24.29  5.655
## 2 white bwt 1021 4990 3076.0 3103.74 727.724
## 3 white lwt  90  235 129.5 132.05 29.094
## 4 black age   15  35  20.5  21.54  5.109
## 5 black bwt 1135 3860 2849.0 2719.69 638.684
## 6 black lwt  98  241 129.0 146.81 39.639
## 7 other age   14  33  22.0  22.39  4.536
## 8 other bwt  709 4054 2835.0 2804.01 721.301
## 9 other lwt   80  250 119.0 120.01 25.130
```

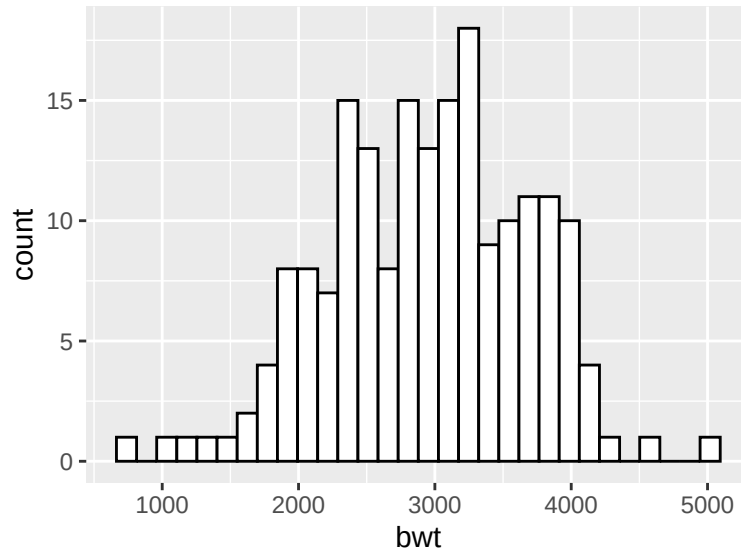
4.4 Biểu đồ tần số và biểu đồ hộp cho 1 biến định lượng

Trong phần này, ta sẽ học cách sử dụng các hàm `geom_histogram()` và `geom_boxplot()`, trong thư viện `ggplot2`, để biểu đồ tần số và biểu đồ hộp cho một biến định lượng.

Cấu trúc lệnh cho hàm `geom_histogram()` là tương tự như cấu trúc của `geom_bar()`. Cụ thể, nếu ta muốn vẽ biểu đồ tần số cho cân nặng của trẻ sơ sinh, `bwt`, trong bộ dữ liệu `data_birth`, một cách đơn giản, ta thực hiện như sau:

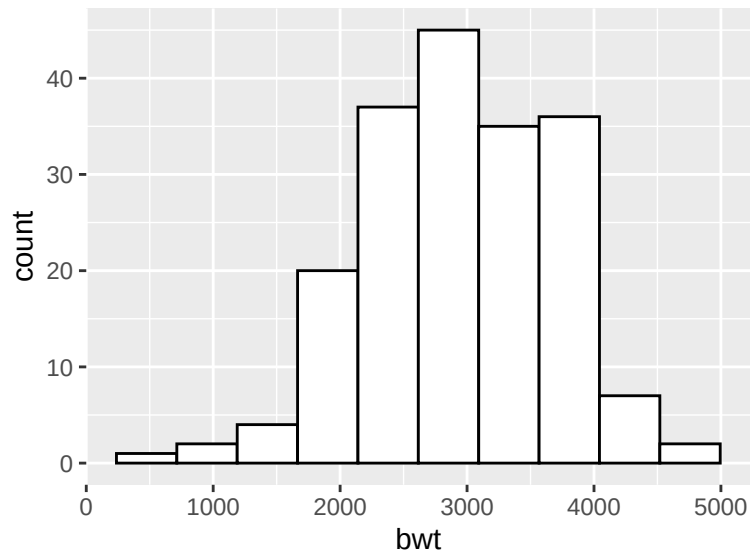
```
ggplot(data = data_birth, aes(x = bwt)) +
  geom_histogram(fill = "white", color = "black")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Theo mặc định, hàm `geom_histogram()` sẽ tự động tính số khoảng tần số (thông qua hàm `stat_bin()`), kết quả thường bị quá nhiều khoảng, dẫn tới biểu đồ không được dễ nhìn (do quá chi tiết). Để khắc phục điều này, ta có thể điều chỉnh số khoảng tần số bằng tham số `bins`, chẳng hạn, `bins = 10`:

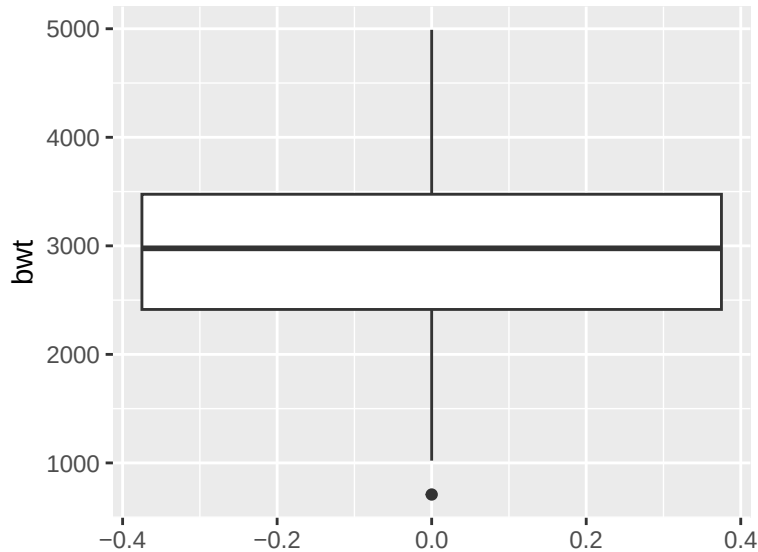
```
ggplot(data = data_birth, aes(x = bwt)) +  
  geom_histogram(fill = "white", color = "black", bins = 10)
```



Ngoài ra, ta cũng có thể dùng đối số `binwidth` để điều chỉnh độ rộng của từng khoảng tần số thay vì điều chỉnh số lượng.

Tương tự, ta có thể vẽ biểu đồ boxplot như sau:

```
ggplot(data = data_birth, aes(y = bwt)) +  
  geom_boxplot()
```



Chú ý, nếu ta muốn có được biểu đồ boxplot nằm ngang, ta có thể sử dụng thêm hàm `coord_flip()`.

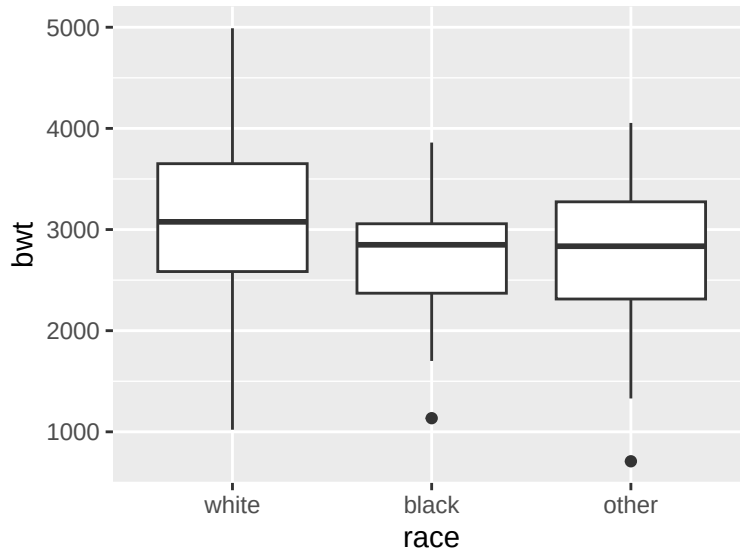
Thực hành 7: thực hiện các thao tác sau:

- vẽ lại biểu đồ tần số và biểu đồ boxplot cho biến `bwt`, thay đổi tên các trục số, hiệu chỉnh màu và hình nền;
- sử dụng hàm `geom_vline()` hoặc hàm `geom_hline()` để tạo một đường thẳng biểu thị giá trị trung bình của `bwt`, chỉnh màu đường thẳng và kiểu đường thẳng, bằng hai đối số `linetype` và `color`;
- vẽ biểu đồ tần số và biểu đồ boxplot cho biến `age` trong dữ liệu `data_birth`, thay đổi tiêu đề tên trục số và màu sắc của biểu đồ;
- sử dụng hàm `geom_vline()` hoặc hàm `geom_hline()` để tạo một đường thẳng biểu thị giá trị trung bình của `age`, chỉnh màu đường thẳng và kiểu đường thẳng;

4.5 Biểu đồ hộp cho 1 biến định lượng phân theo nhóm

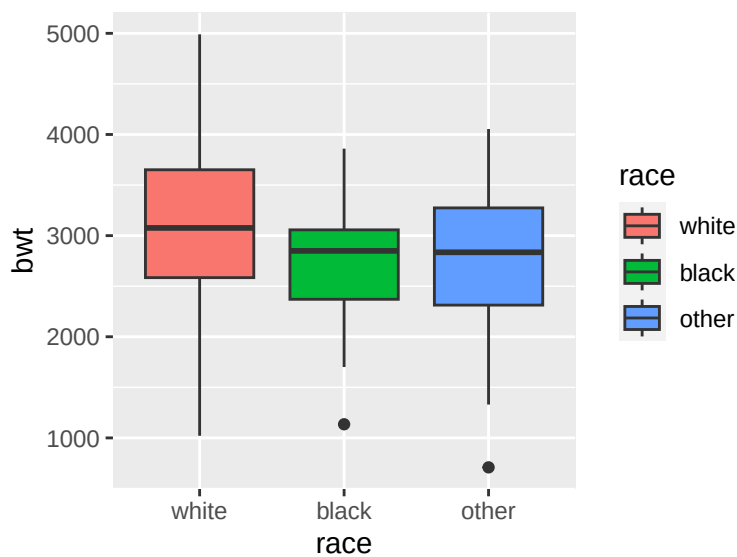
Khi ta muốn so sánh sự khác biệt về giá trị của một biến định lượng, trong các nhóm khác nhau - được xác định bởi một biến định tính, ta có thể sử dụng biểu đồ boxplot. Cú pháp lệnh vẽ là tương tự, tuy nhiên ta sẽ cần thêm `x` bên trong đối số `aes(...)` để xác định biến định tính (phải là được định dạng nhân tố - *factor*). Ví dụ, ta muốn so sánh cân nặng của trẻ sơ sinh, `bwt`, theo ba nhóm chủng tộc của người mẹ (`race`):

```
ggplot(data = data_birth, aes(x = race, y = bwt)) +  
  geom_boxplot()
```



Ta có thể tô màu cho các hộp theo từng nhóm, với `fill = x`, `x` là tên của biến định tính xác định nhóm:

```
ggplot(data = data_birth, aes(x = race, y = bwt, fill = race)) +  
  geom_boxplot()
```



Chú ý: Để bật tắt chú thích của biểu đồ, ta có thể sử dụng đối số `legend.position = "none"` bên trong hàm `theme()`. Xem trang trợ giúp của hàm `theme()` để biết thêm các đối số khác hiệu chỉnh bố cục của biểu đồ.

Thực hành 8: thực hiện các thao tác sau

- vẽ lại biểu đồ boxplot cho biến `bwt` theo `race`, thay đổi tên các trục số, hiệu chỉnh màu và hình nền;
- ghép nối đoạn lệnh vẽ biểu đồ cột phía trên với hàm `facet_wrap(~ smoke)` để tách thành hai biểu đồ con theo trạng thái hút thuốc của người mẹ; ta có thể nhận xét gì dựa trên biểu đồ vẽ về;
- vẽ lại biểu đồ boxplot cho biến `bwt` theo `race`, nhưng dùng thêm tham số `fill = smoke`; so sánh kết quả thu được ở câu trên, khi sử dụng `facet_wrap()`.

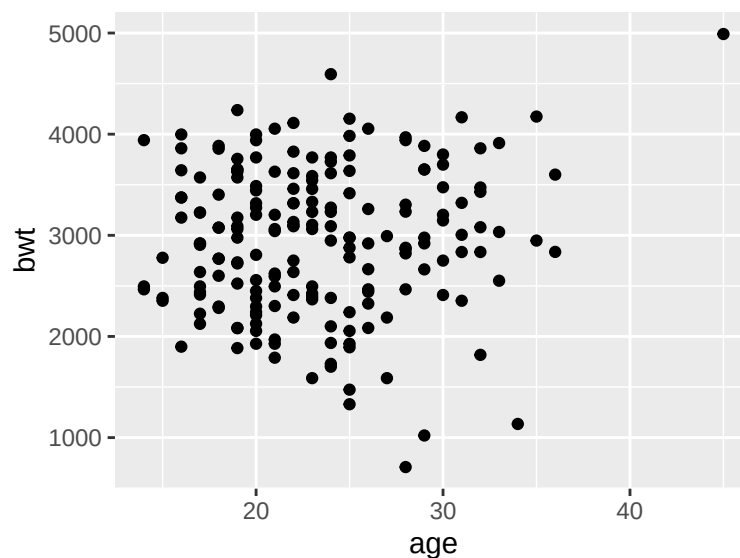
- tạo biến định tính miêu nhóm tuổi của mẹ dựa trên `age`; vẽ biểu đồ boxplot miêu tả cân nặng của trẻ theo nhóm tuổi của mẹ;
- lập lại biểu đồ boxplot trên, nhưng có thêm nhân tố trạng thái hút thuốc của mẹ;
- dùng biểu đồ boxplot để miêu tả thay đổi cân nặng của trẻ sơ sinh theo 3 biến định tính: nhóm tuổi của mẹ, chủng tộc của mẹ, trạng thái hút thuốc.
- sử dụng dữ liệu `MAO.csv`, hãy vẽ biểu đồ boxplot miêu tả sự thay đổi của chỉ số MAO và nhóm bệnh nhân, thực hiện một phiên bản với màu của hộp thay đổi theo nhóm bệnh nhân.

4.6 Biểu đồ phân tán

Biểu đồ phân tán thường được sử dụng để mô tả sự tương quan (mối liên hệ) giữa hai biến định lượng, hoặc cũng có thể là sự tương quan giữa hai biến định lượng và một biến định tính.

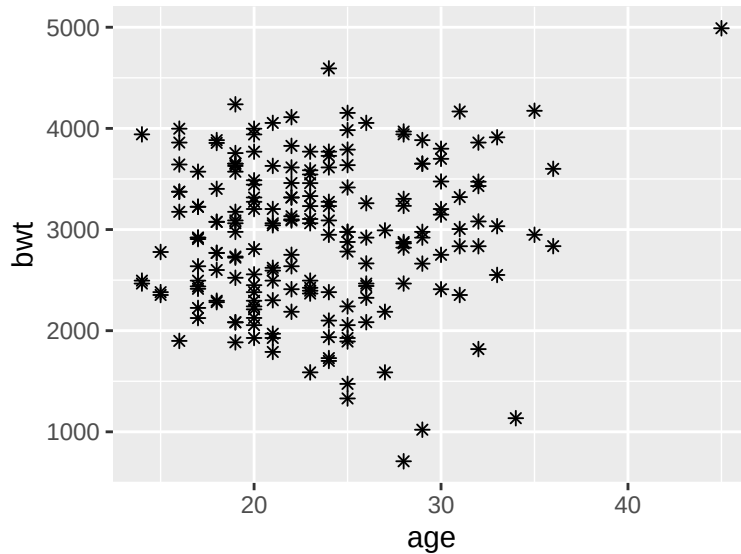
Để vẽ biểu đồ phân tán (*scatter plot*) với thư viện `ggplot2`, ta sử dụng hàm `geom_point()`. Cú pháp lệnh vẽ sẽ tương tự các phần trên. Ví dụ, ta muốn vẽ biểu đồ phân tán biểu diễn sự thay đổi của cân nặng của trẻ sơ sinh (`bwt`) theo tuổi của mẹ (`age`):

```
ggplot(data = data_birth, aes(x = age, y = bwt)) +  
  geom_point()
```



Theo mặc định, hàm `geom_point()` sẽ vẽ các điểm dạng hình tròn màu đen, tuy nhiên ta có thể đổi dạng của điểm bằng đối số `shape` bằng các giá trị từ 1 tới 25. Ví dụ:

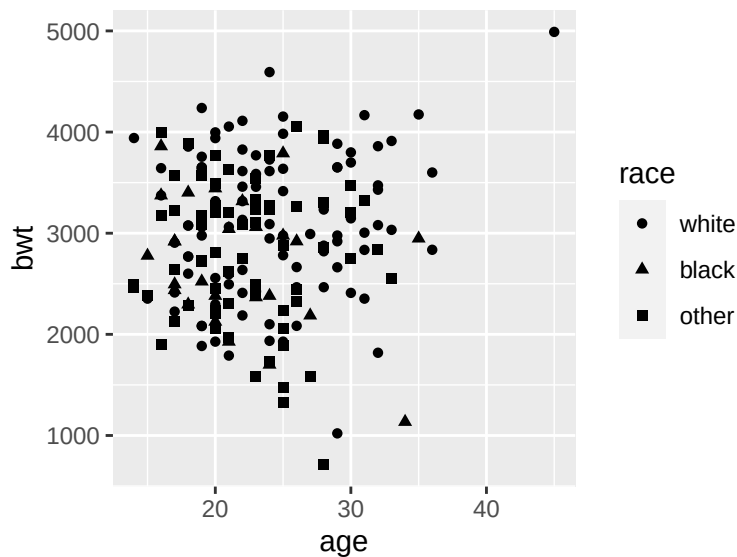
```
ggplot(data = data_birth, aes(x = age, y = bwt)) +  
  geom_point(shape = 8)
```



Ngoài ra, ta cũng có thể đổi màu của các điểm bằng đối số `color`.

Trong trường hợp, ta muốn khám phá mối liên hệ của cân nặng của trẻ sơ sinh và độ tuổi, theo nhóm chủng tộc của mẹ (một biến định tính), ta có thể sử dụng `shape = race` nhằm tạo ra các kiểu hình khác nhau cho các điểm dữ liệu dựa trên thông tin nhóm chủng tộc:

```
ggplot(data = data_birth, aes(x = age, y = bwt, shape = race)) +  
  geom_point()
```



Có thể dùng thêm màu sắc, hoặc hàm `facet_wrap()` nếu ta muốn làm với nhiều hơn một biến định tính.

Thực hành 9: thực hiện các thao tác sau

- vẽ lại biểu đồ phân tán cho biến `bwt` theo `age`, thay đổi tên các trục số, hiệu chỉnh màu, hình dạng của điểm và hình nền, sau đó, hãy thử với log của cân nặng;
- ghép nối đoạn lệnh vẽ biểu đồ cột phía trên với hàm `facet_wrap(~ smoke)` để tách thành hai biểu đồ con theo trạng thái hút thuốc của người mẹ; ta có thể nhận xét gì dựa trên biểu đồ vẽ vẽ;

- vẽ lại biểu đồ phân tán cho biến `bwt` theo `age`, nhưng dùng thêm tham số `color = smoke`; so sánh kết quả thu được ở câu trên, khi sử dụng `facet_wrap()`.
- dùng biểu đồ boxplot để miêu tả thay đổi cân nặng của trẻ sơ sinh theo 3 biến định: tuổi của mẹ, chủng tộc của mẹ, trạng thái hút thuốc.

5 Xuất file hình

Hình vẽ biểu đồ trong R/Rstudio, vào một tệp định dạng hình ảnh như .png, .jpg hoặc định dạng vector như .pdf, .eps. Có nhiều hàm khác nhau để thực hiện việc xuất/lưu trữ hình vẽ trong R, tuy nhiên, lựa chọn tốt nhất là các cách lưu dựa trên cairo graphics API, vì giúp lưu được các tiêu đề bằng tiếng Việt.

Để chất lượng hình lưu không bị ảnh hưởng khi thu phóng hình, ta nên dùng định dạng cuối là .pdf, định dạng này có thể dán được trong văn bản khi soạn thảo bằng latex. Tuy nhiên, nếu sử dụng word thì ta nên sử dụng định dạng cuối là .png. Cụ thể:

- `cairo_pdf(filename = "ten_hinh.pdf", width = 6, height = 5)`: dùng để xuất hình dưới dạng tệp .pdf, trong đó, `width` và `height` lần lượt là độ rộng và chiều cao của hình, được tính theo inch, ảnh hưởng tới độ thu phóng của hình;
- `png(filename = "ten_hinh.png", width = 480, height = 480, type = "cairo")`: dùng để xuất hình dưới dạng tệp .png, trong đó, `width` và `height` được tính theo pixel.
- `jpeg(filename = "ten_hinh.jpg", width = 480, height = 480, quality = 100, type = "cairo")`: dùng để xuất hình dưới dạng tệp .jpg

Cấu trúc đoạn lệnh như sau:

```
cairo_pdf(filename = "ten_hinh.pdf", width = 6, height = 5)
\\ đoạn lệnh vẽ hình
dev.off()
```

Ví dụ, với biểu đồ phân tán miêu tả sự thay đổi của cân nặng trẻ sơ sinh theo độ tuổi của người mẹ. Ta có thể lưu lại theo các định dạng file khác nhau như sau:

```
cairo_pdf(filename = "bieu_do_phan_tan_1.pdf", width = 6, height = 5)
ggplot(data = data_birth, aes(x = age, y = bwt)) +
  geom_point() +
  xlab("Tuổi") + ylab("C\u00e2n nặng của trẻ sơ sinh") +
  theme_classic()
dev.off()
```

```
png(filename = "bieu_do_phan_tan_1.png", width = 480, height = 480,
     type = "cairo")
ggplot(data = data_birth, aes(x = age, y = bwt)) +
  geom_point() +
  xlab("Tuổi") + ylab("C\u00e2n nặng của trẻ sơ sinh") +
  theme_classic()
dev.off()
```

```
jpeg(filename = "bieu_do_phan_tan_1.jpg", width = 480, height = 480,
      quality = 100, type = "cairo")
ggplot(data = data_birth, aes(x = age, y = bwt)) +
  geom_point() +
  xlab("Tuổi") + ylab("C\u00e2n nặng của trẻ sơ sinh") +
  theme_classic()
dev.off()
```